

STUDENT EXPENSE TRACKER SYSTEM

Programming in Java - Project Report

Student Name	Simone Chouksey
Registration No.	25 BAR 10006
Course	Programming in Java
Course Code	CSE2006
Project Type	Build Your Own Project

A command-line Java application for recording, storing and analysing student expenses.

1. Introduction

The Student Expense Tracker System is a command-line Java application intended to help students maintain a simple record of daily expenses. The project applies core Java concepts in a practical context, including object-oriented design, collections, exception handling and file I/O.

2. Objectives

- Record and manage expenses.
- Categorise expenses for easier tracking.

- Generate basic spending summaries.
- Persist expense data using file handling.
- Apply Java OOP concepts in a modular application.

3. Problem Statement

Students often record expenses informally and may lose track of total spending or spending categories. This project provides a lightweight terminal-based solution that keeps expense records in one place and produces simple summaries.

4. Scope

The system focuses on local command-line expense management. It does not require an external database or graphical interface. Future versions can add JDBC, a GUI and richer analytics.

5. Functional Requirements

Module	Functions
Expense Management	Add, view, update and delete expenses
Category Management	Assign categories such as Food, Travel, Education and Entertainment
Reporting	Calculate total spending, category-wise totals and highest expense
Persistence	Save and load expense records from a text file

6. Non-Functional Requirements

- Performance - normal operations should complete quickly for a student-scale dataset.
- Reliability - invalid input and file errors should be handled without crashing the program.
- Usability - menu-driven commands should be clear to a first-time user.
- Maintainability - classes and responsibilities should remain separated and easy to update.

7. Technical Concepts Used

- Classes and objects
- Encapsulation
- Inheritance and interface-based design
- Polymorphism
- ArrayList / Collections Framework

- Exception handling

- Character-based file I/O

8. System Architecture

The application separates user interaction, expense management, reporting and persistence. This keeps each component focused on one responsibility.

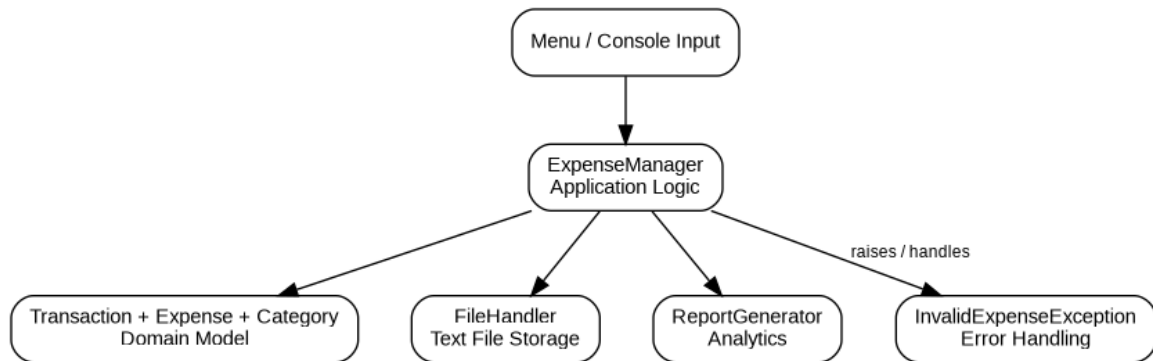


Figure 1. High-level system architecture

9. Design Diagrams

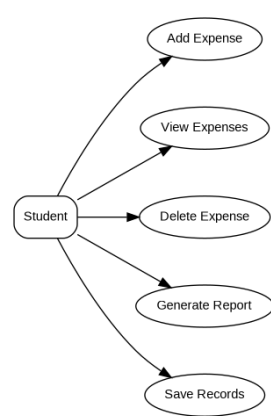


Figure 2. Use case diagram

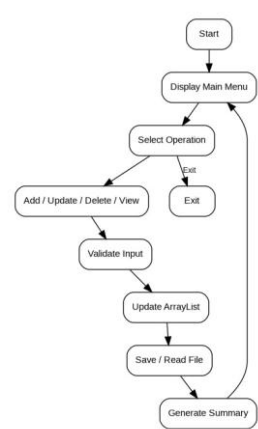


Figure 3. Workflow diagram

10. Class Diagram

The class design uses encapsulation for expense data, an interface for printable behaviour, and service classes for management, persistence and reporting.

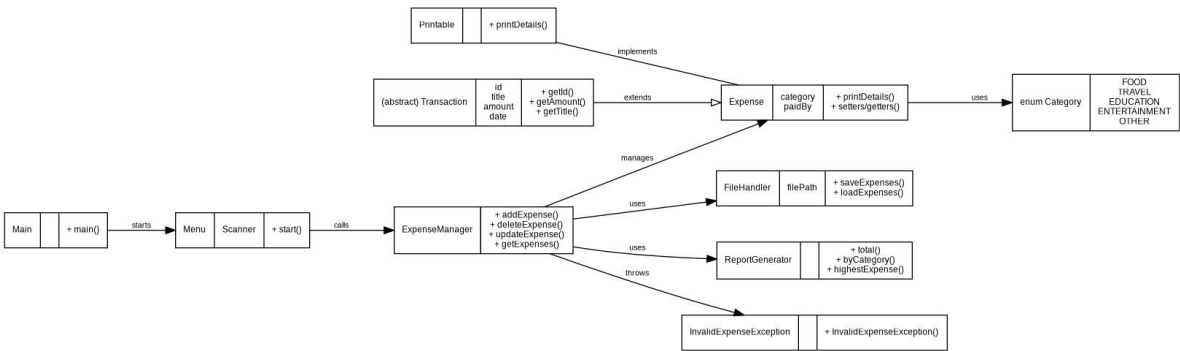


Figure 4. Class diagram

11. Sequence Diagram

The sequence below shows a typical interaction in which the user adds an expense, the manager stores it and the file handler persists it.

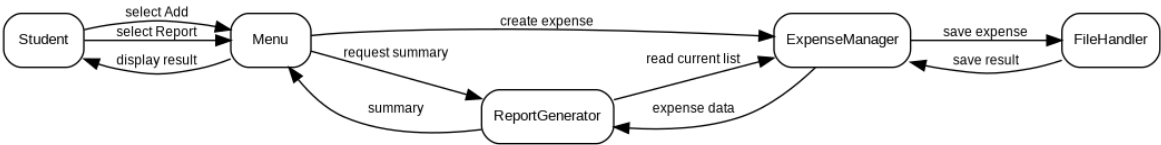


Figure 5. Sequence diagram

12. Implementation Details

Expense.java - represents an individual expense and stores fields such as id, title, amount and category.

ExpenseManager.java - performs add, update, delete and view operations using an ArrayList.

Category.java - defines available expense categories.

FileHandler.java - saves and loads expense records using character-oriented file I/O.

ReportGenerator.java - calculates totals and category-wise summaries.

InputValidator.java - validates user input and helps prevent invalid values.

Menu.java / Main.java - provide the command-line interface and program entry point.

13. Testing Approach

Test Case	Expected Result
Add valid expense	Expense is added to the list
Enter invalid amount	Input is rejected and user is prompted again
View expenses	Stored records are displayed
Delete existing expense	Selected expense is removed
Generate report	Totals and summaries are displayed
Save and reload	Records remain available after restart

14. Challenges Faced

- Designing the classes so each class has a clear responsibility.

- Handling invalid console input and exceptions.
- Keeping stored data consistent while reading and writing files.

15. Learnings and Key Takeaways

- Applied encapsulation, inheritance, interfaces and polymorphism in a small application.
- Practised Java collections and file I/O.
- Improved understanding of exception handling and modular program design.

16. Future Enhancements

- JDBC-based database storage.
- Graphical user interface.
- Monthly and trend-based analytics.
- Export of reports to CSV or PDF.

17. Conclusion

The Student Expense Tracker System provides a practical demonstration of core Java programming concepts through a small, modular command-line application. Its design can be extended with database and user-interface features while keeping the same core services.

18. References

- CSE2006 Programming in Java course syllabus provided by VIT Bhopal.
- Herbert Schildt, Java: The Complete Reference, 11th Edition, Oracle Press.

- Oracle Java documentation and API references.